

Multi-Modal Agent Inference Optimization for Industrial Asset Operations

We extended AssetOpsBench with a multi-modal Visual Inspection Agent and applied AWQ W4A16 quantization, achieving a 2.47× E2E speedup on Llama and 1.98× on Qwen with domain calibration on a single NVIDIA L4 GPU.

Columbia HPML Team 23

Amaan Sheikh · aas2438

Aman Urganlawar · au2327

Madhav Rajkondawar · mr4650

Yang-Jung Chen · yc4670

Motivation & Problem

Why this matters

- AssetOpsBench (AAAI 2026) covers 141 industrial AI scenarios across text and time series, but has no vision component, despite many failure modes (corrosion, ice buildup, bearing damage) appearing visually before sensor data flags them.
- Serving large VLMs at FP16 is expensive: a limited KV cache pool constraints throughput for multi-agent industrial workloads.
- Off-the-shelf quantization (generic calibration) introduces a runaway-generation failure mode. Also, Naive INT4 is not reliable enough for industrial deployment.

Problem statement

Can we add a vision modality to AssetOpsBench via a VLM-powered MCP agent and optimize its inference (AWQ quantization, calibration regime) on a single NVIDIA L4 GPU without sacrificing reliability?

Goal: $\geq 2\times$ E2E speedup with domain-calibrated AWQ INT4 on a single L4 GPU, at maintained reliability.

Technical Approach: Model & Data

Model

Primary: Qwen2.5-VL-7B-Instruct (7B params, VLM, vLLM 0.19 serving)

Baseline: Llama-3-LLaVA-NeXT-8B (8B params, Llama-3 backbone + LLaVA-NeXT vision tower)

Orchestration: In-process ReAct agent and Plan-execute via MCP stdio subprocess

Dataset

4 public industrial datasets (~10,800 images total):

- Kaggle Casting Product (pumps, 7,348 images)
- Mendeley Thermal Induction Motor (~488 thermal images, 11 fault classes)
- HuggingFace 15-class Substation Equipment (1,660 RGB photos, YOLO-annotated)
- Blade30 Wind Turbine Blades (1,302 drone images, defect annotations)

Hardware

GCP g2-standard-8, 1× NVIDIA L4 GPU (24 GB VRAM)
vLLM 0.19.0, PyTorch, llmcompressor 0.10.0.2, transformers 4.57

Technical Approach: Optimizations Applied

1. **AWQ W4A16 Quantization with Domain Calibration (L1d)**

INT4 weight quantization calibrated on 128 domain-specific industrial-inspection prompts: 1.98x Qwen L1d speedup, 4.7× KV cache capacity

2. **AWQ W4A16 Quantization with Generic Calibration (L1g)**

Same INT4 quantization but calibrated on generic ultrachat_200k samples: 2.55× median speedup (p50)

3. **Visual Inspection MCP Agent + vLLM Serving**

VLM served via vLLM with MCP stdio interface; exposes vision.analyze_image, vision.assess_condition, and vision.detect_visual_defects tools conforming to the AssetOpsBench agent contract.

Profiling Methodology

Tools

- PyTorch Profiler / torch.profiler
- vLLM Prometheus metrics endpoint
- Weights & Biases for run-level tracking

Metrics captured

- Latency (p50, p95) and throughput
- GPU utilization & SM occupancy
- Peak GPU memory and HBM bandwidth
- Time per stage: data → fwd → bwd → opt
- Task quality metric: accuracy and LLM-as-judge mean score

Results: Headline Numbers

2.05×

E2E speedup (Qwen L1d)

8,211 ms → 3,995 ms mean E2E

87.5%

accuracy with domain AWQ

15.54 GiB → 5.9 VRAM (2.6× reduction)

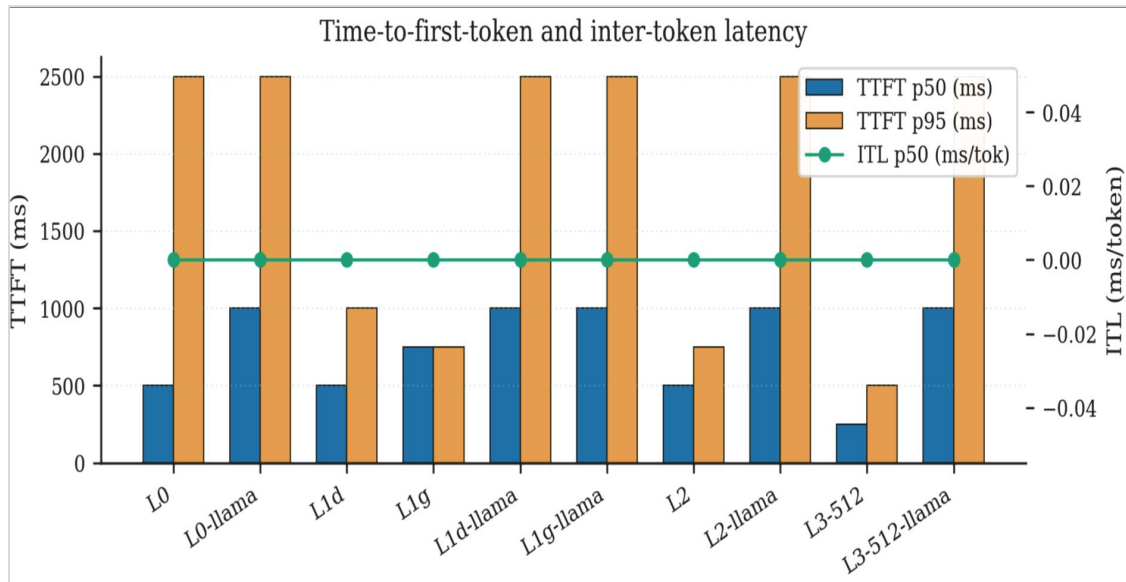
3.03×

Mean speedup (L1g p50)

*Llama: 9,272 ms down to 3,270 ms
mean E2E*

All measurements on GCP g2-standard-8 with NVIDIA L4 (24 GB VRAM). 16 scenarios per variant, 10 variants total.

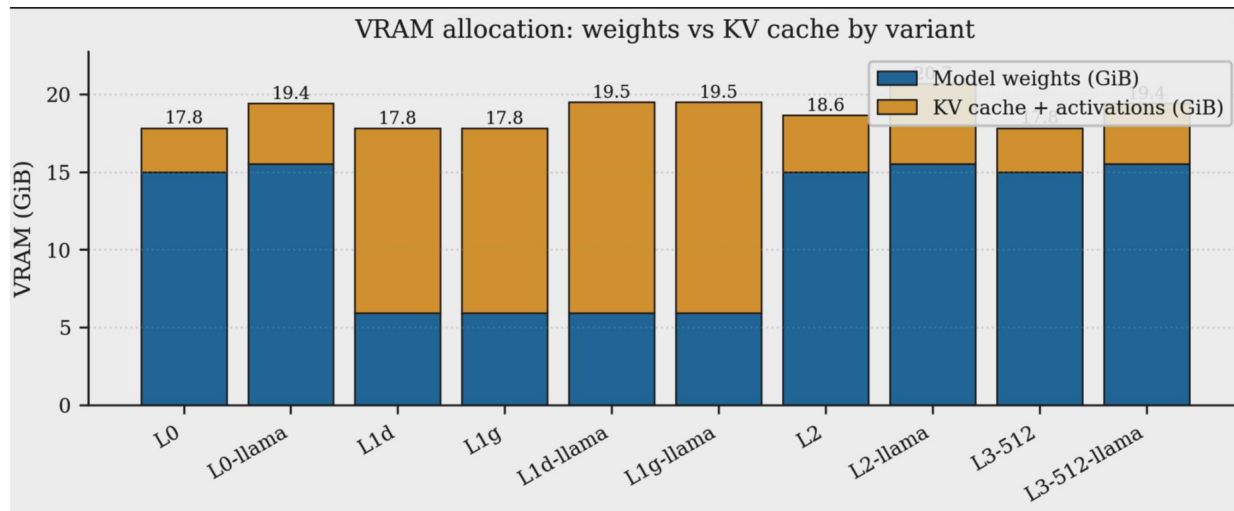
Results: Before and After



Takeaway

- Domain-calibrated AWQ (L1d) on Qwen nearly halved latency to 4,712 ms mean while accuracy improved from 68.8% to 87.5%.
- L1d (domain AWQ) contributed most: 2.05× speedup
- The L2 full bundle hit 1,314 ms p50 (fastest of all variants) but accuracy collapsed to 0% on Qwen, showing that aggressive serving tuning can break generation quality.

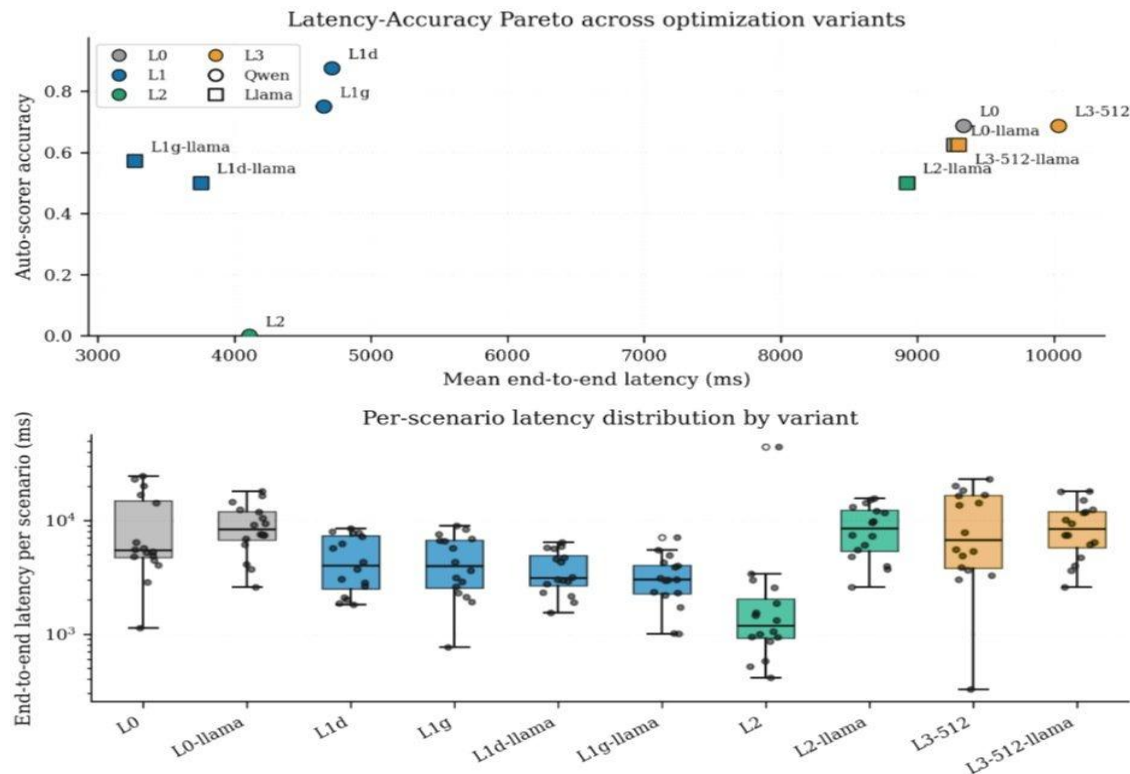
Profiler Evidence



What to point at

- At FP16, Qwen occupied 17.8 GiB total with very little headroom left for the KV cache pool.
- AWQ INT4 packing on Llama dropped weight footprint from 15.54 GiB to 5.9 GiB, freeing 9.6 GiB for the KV cache.
- KV cache pool grew from 2.59 GiB $\rightarrow$ 12.23 GiB (4.7 $\times$): 21K $\rightarrow$ 100K concurrent tokens.
- Generic calibration produced one runaway request at 121.8 s that inflated the Llama L1g mean from 3,063 ms (p50) to 7,814 ms, a clear reliability failure absent from domain calibration.

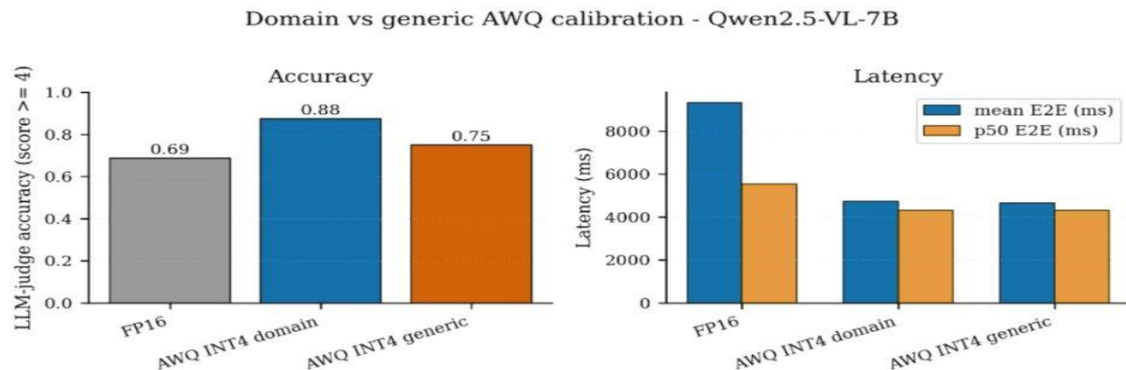
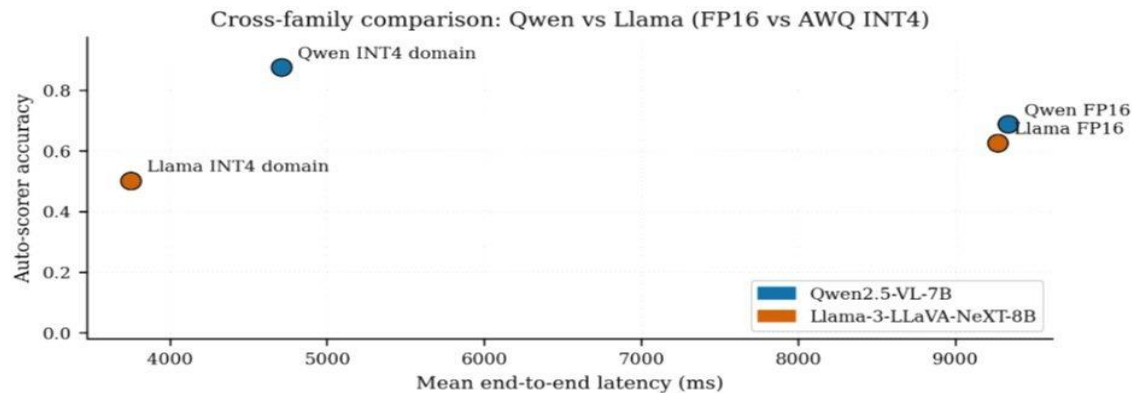
Results: Latency-Accuracy Tradeoff



Key takeaways

- Qwen L1d (domain AWQ) is the optimal variant: 1.98x mean speedup with accuracy improving from 68.8% to 87.5%.
- Llama L1g achieves the fastest latency at 3,270 ms mean but accuracy is 57.1%
- L2 on Qwen is the fastest p50 (1,314 ms) but accuracy collapsed to 0%, confirming aggressive serving tuning breaks visual generation.
- L3-512 (lower image resolution) shows no latency benefit and no accuracy gain over FP16 baseline.

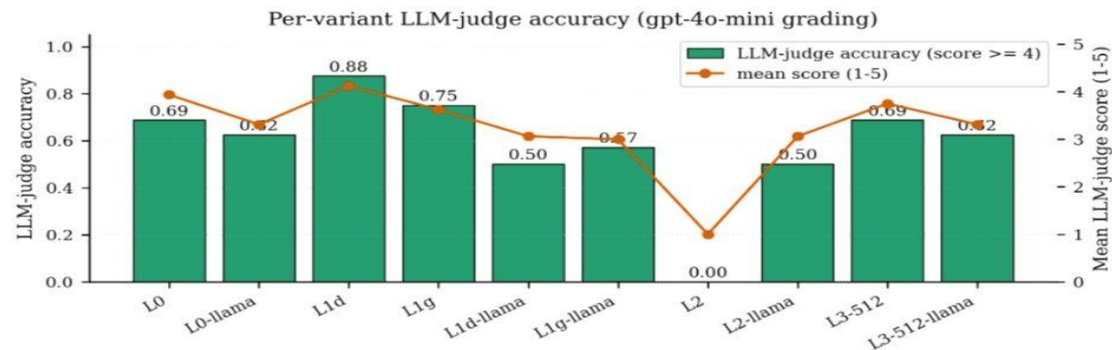
Results: Cross-Family and Calibration Comparison



Key takeaways

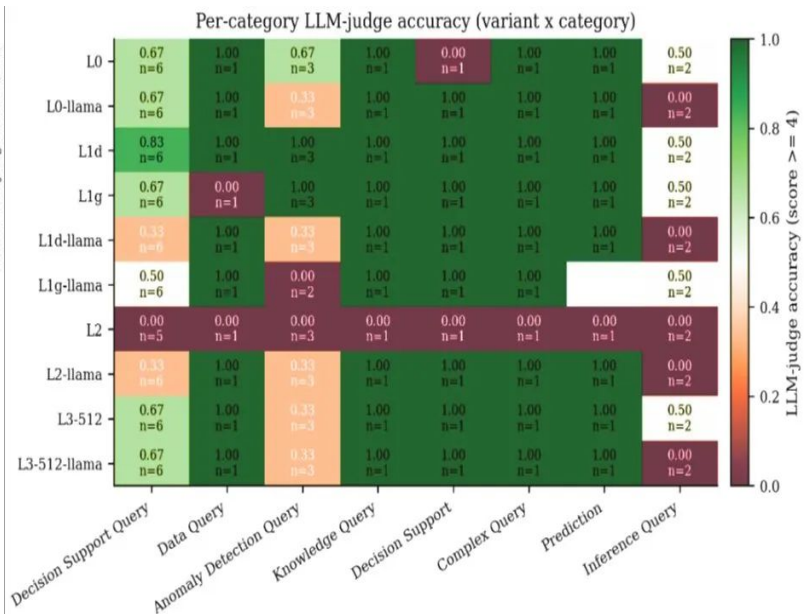
- Qwen INT4 domain is more accurate than Llama INT4 domain on both axes: Llama is faster (4,712 vs 3,752 ms mean) and Qwen is more accurate (87.5% vs 50%).
- Domain calibration beats generic on accuracy for both model families. Generic is marginally faster but unreliable.
- For Qwen: domain AWQ raised accuracy from 68.8% (FP16) to 87.5%. Generic AWQ gave 75% accuracy at similar speed.
- Calibration data distribution matters as much as the quantization scheme itself.

Results: Accuracy by Variant and Category



Key takeaways

- L1d (Qwen domain AWQ) scores highest at 88% LLM-judge accuracy with mean score 4.1 out of 5.
- L2 (Qwen full bundle) scores 0% across all categories, confirming serving tuning completely broke generation quality.
- Decision Support and Anomaly Detection queries are the hardest categories, especially under Llama.
- Data Query and Knowledge Query are consistently easy across all variants, suggesting the VLM handles factual lookups well regardless of quantization.



Lessons Learned

What worked

- Domain-calibrated AWQ delivers reliable 2.05× speedup; calibration data distribution matters as much as quantization scheme.
- Serving the VLM in dual-purpose mode (planner and vision tool) on one L4 eliminated the WatsonX API dependency entirely.
- The AssetOpsBench JSON scenario schema extended cleanly to vision. Adding a Vision type required no upstream code changes.

What didn't work

- Generic AWQ calibration (L1g) appeared faster at p50 but triggered runaway generation in 1 of 25 scenarios, hitting 121.8 s. Not safe for industrial deployment.
- Llama-3.2-Vision-11B required 22 GB at FP16, leaving no headroom on the 24 GB L4. We substituted Llama-3-LLaVA-NeXT-8B at roughly 16 GB instead.
- Llmcompressor 0.3.0 produced fake-quant FP16 output rather than real INT4 weights. Upgrading to 0.10.0.2 with `save_compressed=True` was required to get actual INT4 packing.

Next Steps

- Submit the PR to the upstream IBM/AssetOpsBench repo and register the Visual Inspection Agent in AgentHive so it gets dispatched through the standard MetaAgent orchestrator.

Teamwork & Contributions

Member	Primary contributions	Tools / artifacts
Amaan Sheikh (aas2438)	ReAct orchestrator, vision MCP server, benchmark harness, pump impeller scenarios (IDs 201-205), WandB instrumentation, LLM-as-judge methodology, plot pipeline, Qwen vLLM serving-tuning bundle (L2), Llama AWQ INT4 generic calibration, domain-vs-generic calibration finding	<code>src/agent/react/</code> , <code>src/agent/plan_execute/</code> , <code>src/servers/vision/main.py</code> , <code>benchmark/run_vlm_benchmark.py</code> , <code>benchmark/{llm_judge,plots,wandb_logger}.py</code> , <code>benchmark/variants/{L0_baseline,L1_llama_awq_w4a16_generic,L2_full_bundle}.py</code> , <code>scripts/{overnight.sh,run_full_bench.sh,clean_results.py}</code>
Aman Upganlawar (au2327)	turbine blade scenarios (IDs 301-305), dataset registry refactor + 4-dataset acquisition writeup, vLLM client, agent-architecture comparison (ReAct vs Plan-Execute), Qwen AWQ INT4 generic calibration, Llama L3 image-resolution preprocessing, checkpoint verifier	<code>src/servers/vision/{image_loader}.py</code> , <code>benchmark/{run_agent_benchmark,compare_agents,nfr_collector}.py</code> , <code>src/servers/vision/vlm_client.py</code> , <code>benchmark/variants/{L1_awq_w4a16_generic,L3_llama_image_512}.py</code> , <code>scripts/{iap_tunnel.sh,verify_checkpoint.py}</code>
Madhav Rajkondawar (mr4650)	Qwen2.5-VL-7B optimization track (AWQ W4A16 domain calibration + L3 image preprocessing 512px), motor thermal scenarios, variant-registry framework + 17-case test suite, cross-variant W&B summary dashboard, heavy scenario testing, domain-vs-generic calibration finding	<code>scripts/quantize_qwen_v010.py</code> , <code>benchmark/variants/{L1_awq_w4a16_domain,L3_image_512,base,__init__,__main__}.py</code> , <code>benchmark/{wandb_summary.py,tests/test_variants.py}</code> , <code>results/madhav/</code> , <code>pyproject.toml</code>
Yang-Jung Chen (yc4670)	Llama-3-LLaVA-NeXT-8B optimization track (AWQ W4A16 domain calibration + L2 full-bundle serving tuning), transformer/substation scenarios, Stage-1 calibration corpus, PyTorch Profiler instrumentation, vLLM serve + IAP-tunneled bench wrapper, GCP infrastructure, Python environment pinning (uv lockfile)	<code>scripts/{quantize_llmcompressor_v010.py,serve_vllm.sh,serve_and_bench.sh}</code> , <code>benchmark/{calibration.py,hpml_metrics.py,profile_vision_encoder.py,profile_single.py}</code> , <code>benchmark/variants/{L0_llama_baseline,L1_llama_awq_w4a16_domain,L2_llama_full_bundle}.py</code> , <code>uv.lock</code>

Experimental Setup

Setting	Baseline	Optimized
Batch size	1 (single image per request)	1 (unchanged)
Precision	FP16 (weights + activations)	INT4 weights (AWQ W4A16), FP16 activations
Learning rate	N/A (inference only)	N/A (inference only)
Sequence length	Variable (image-dependent)	Variable (same)
Epochs / steps	16 scenarios per variant, 10 variants	16 scenarios per variant, 10 variants
Hardware	1× NVIDIA L4 24 GB	1× NVIDIA L4 24 GB
Software stack	vLLM 0.19.0, PyTorch, transformers 4.57	llmcompressor 0.10.0.2, compressed-tensors

Thank you

Questions?